

```

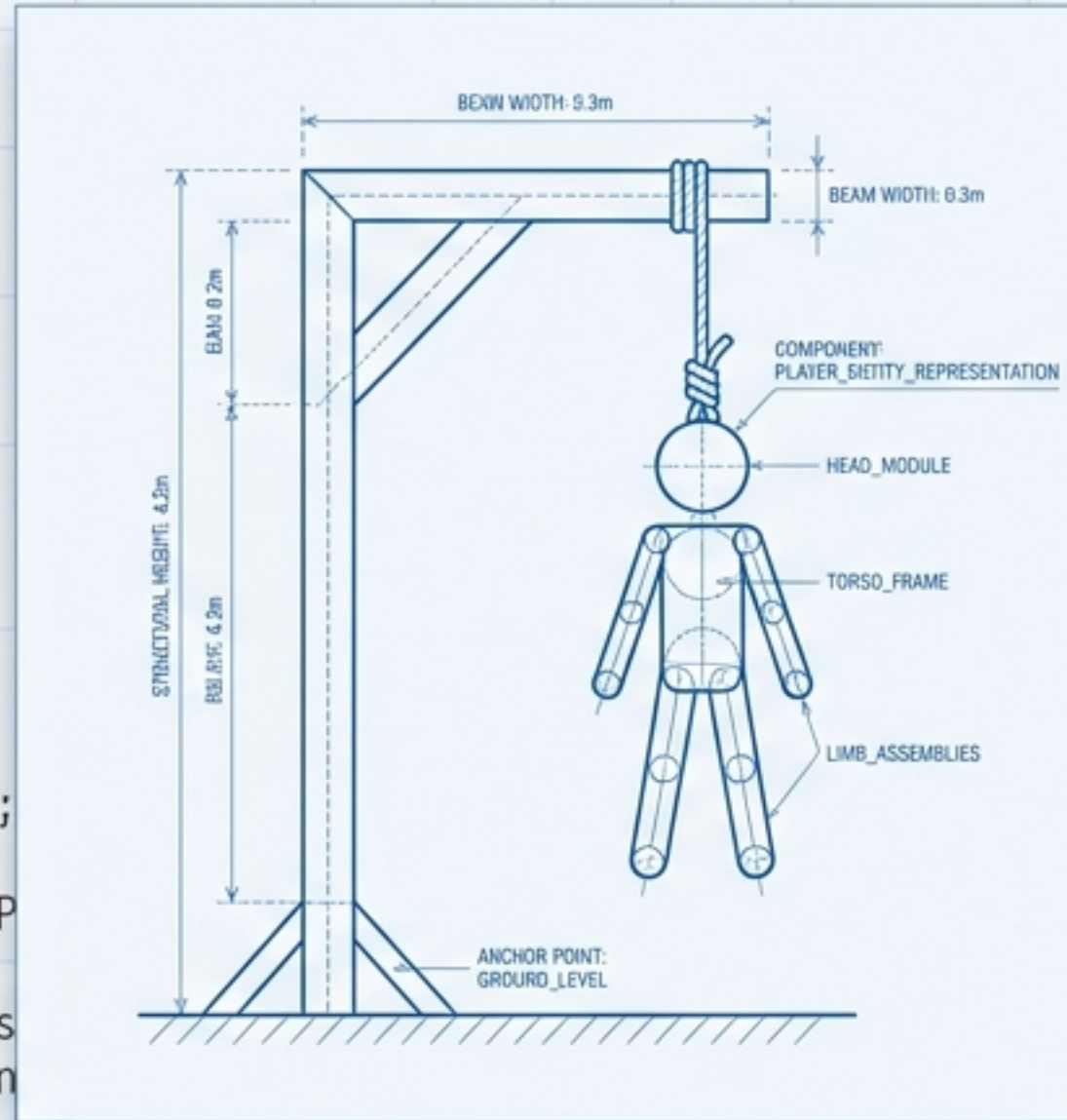
using System;
using System.Data.SQLite;

namespace HangmanGame.Core
{
    public class GameSession
    {
        private string _secretWord;
        private int _attemptsLeft;

        public void StartNewGame()
        {
            /* ... init logic ... */
        }
    }

    public class DatabaseManager {
        private readonly string _connectionString;
        public DatabaseManager(string dbPath) {
            _connectionString = $"Data Source={dbPath}.db";
        }
        public void SaveGameState(GameSession session) {
            using (var connection = new SQLiteConnection(_connectionString)) {
                connection.Open();
                /* ... SQL insert ... */
            }
        }
    }
}

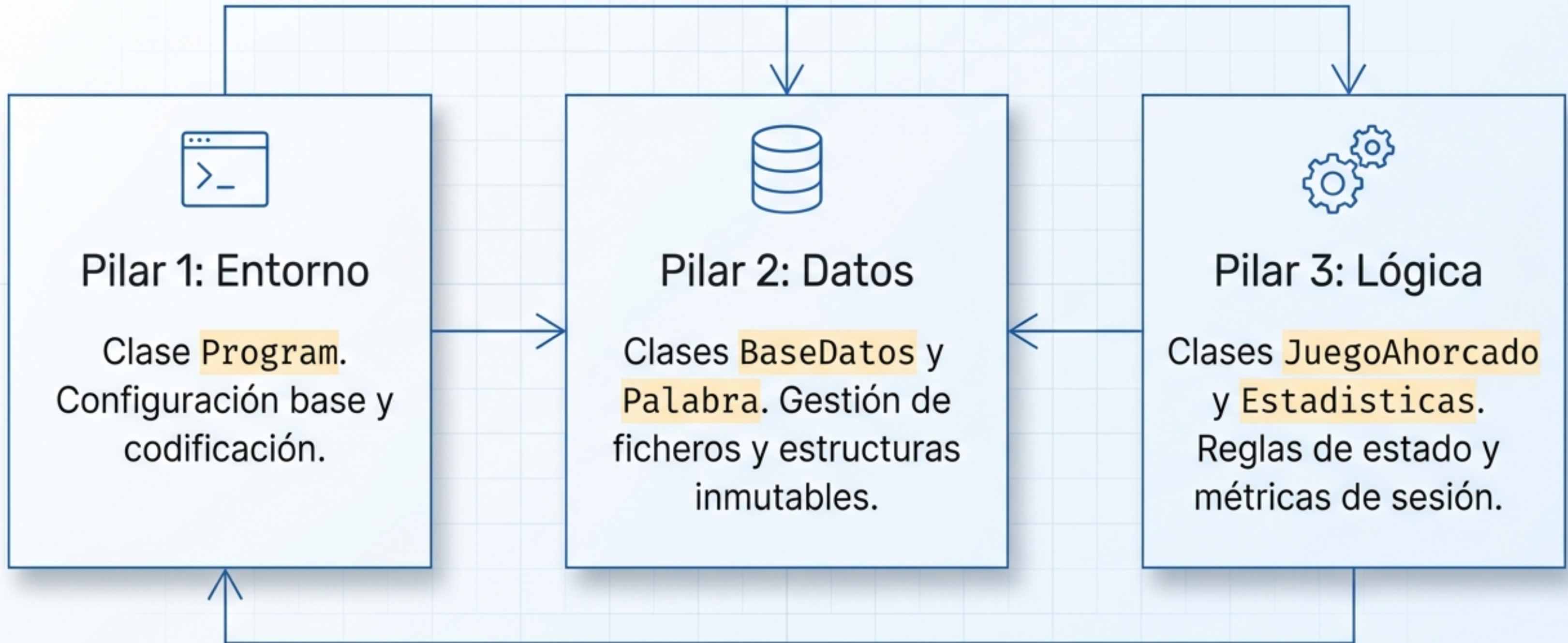
```



Arquitectura del Juego del Ahorcado

Implementación en C# y SQLite paso a paso.

El Plano General: Separación de Responsabilidades



El Entorno: Configuración de Consola

```
using System;  
using System.Text;  
  
class Program {  
    static void Main() {  
        Console.OutputEncoding = Encoding.UTF8;  
    }  
}
```



Canciñ
aadir



Canción
añadir

Algunas consolas no muestran correctamente las tildes y los signos españoles si no se indica expresamente la codificación UTF-8.

La Fundación: Base de Datos Local

```
using System;  
using Microsoft.Data.Sqlite;  
  
class BaseDatos  
{  
    private const string CadenaConexion = "Data Source=ahorcado.db";  
}
```

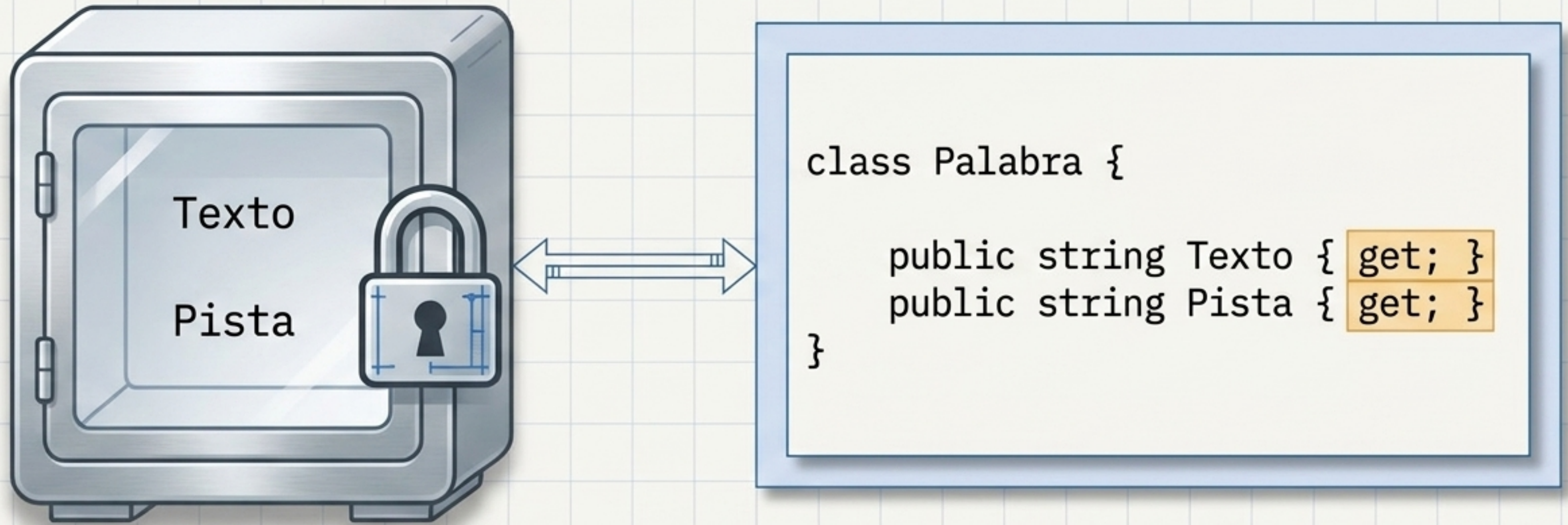
Un único fichero. SQLite guarda toda la base de datos localmente.

Autocreación. Si el fichero no existe, se crea automáticamente al abrir la cadena de conexión.



ahorcado.db

Los Ladrillos: Estructuras de Datos Inmutables



Inmutabilidad. Estas propiedades solo tienen get porque la palabra y la pista no deben cambiar una vez que han sido leídas de la base de datos.

El Motor: Dependencias Seguras

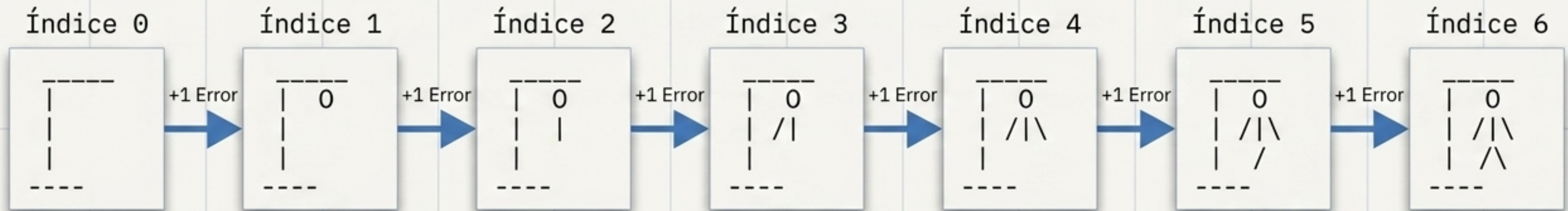
```
using System;  
using System.Collections.Generic;  
class JuegoAhorcado {  
    private readonly BaseDatos baseDatos;  
}
```



Enlace Permanente. El juego guardará esta referencia durante toda su ejecución.

Protección de Estado. readonly impide que se sustituya accidentalmente por otro objeto, garantizando la estabilidad de la conexión.

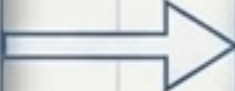
Los Activos Visuales: Evolución del Estado



Un array es una máquina de estados. Cada intento fallido incrementa el índice, llamando al siguiente activo visual en la secuencia.

El Marcador: Métricas de Sesión

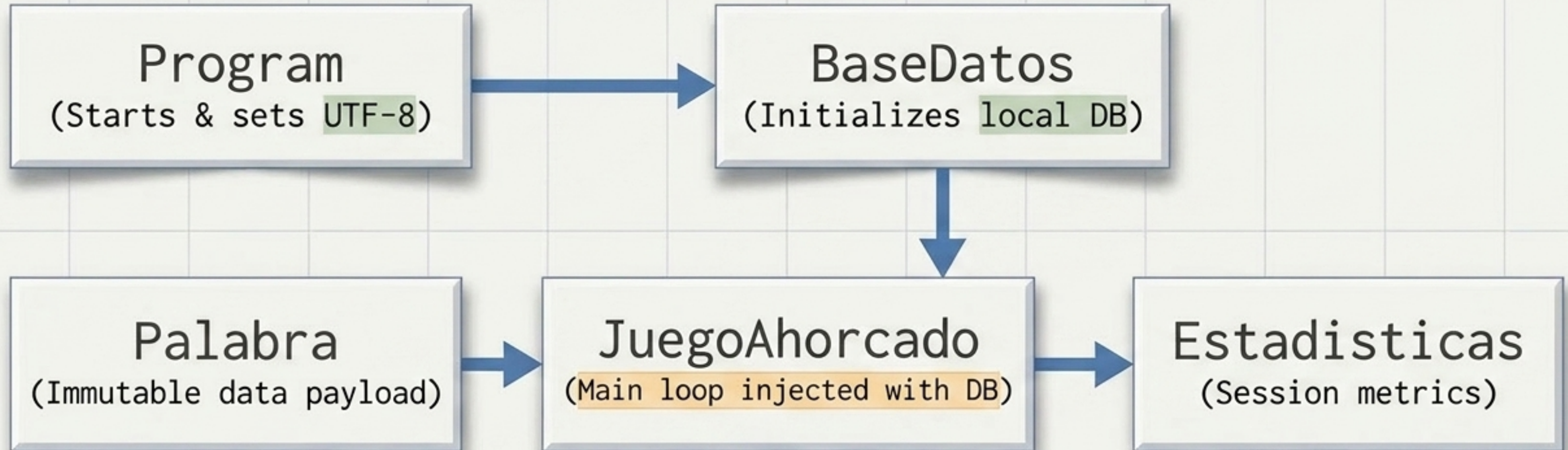
Partidas: 12
Victorias: 8



```
class Estadisticas {  
    public int Partidas { get; }  
    public int Victorias { get; }  
}
```

Principio de Responsabilidad Única. Separar las estadísticas del bucle principal del juego mantiene el código modular y fácil de ampliar en el futuro.

Síntesis: El Ciclo de Vida del Juego



La arquitectura transforma componentes aislados en un flujo de ejecución cohesivo y predecible.

Tu Plano de Desarrollo está Listo

- ✓ 1. Configuración de Entorno (UTF-8) para español de España.
- ✓ 2. Integración de base de datos local con SQLite (Fichero único).
- ✓ 3. Protección de datos con Inmutabilidad (get-only).
- ✓ 4. Inyección de dependencias seguras (readonly).
- ✓ 5. Separación de Clases y Responsabilidades.

Compila este código.

Rompe las reglas.

Añade una interfaz gráfica o nuevas estadísticas.

El código ahora es tuyo.